# NOVAC Internship – Task 9 Daily Documentation

Task: Hierarchical Semantic Chunking Optimization, Groq AI Chunking Pipeline Improvements, Retrieval Accuracy Enhancement, Production-Style RAG Architecture Refinement, and Conversational Context Retrieval Improvements

## Overview

Today's work focused on significantly improving the Retrieval-Augmented Generation (RAG) architecture by optimizing the semantic chunking pipeline and enhancing retrieval quality.

The primary objective was to solve major chunking and retrieval problems encountered during previous implementations, including:

- heading-only chunk generation
- giant full-document chunks
- broken semantic boundaries
- retrieval inconsistency
- poor contextual chunk segmentation
- duplicate storage conflicts
- malformed chunk fragments

The system was redesigned into a more production-style hierarchical semantic chunking architecture using Groq-powered semantic segmentation combined with SentenceTransformer embeddings and MongoDB vector persistence.

By the end of today's implementation, the system successfully supported:

- hierarchical semantic chunking
- paragraph-aware preprocessing
- Groq AI-powered semantic segmentation
- improved retrieval precision
- overwrite-based chunk updating
- conversational contextual retrieval
- cleaner chunk persistence workflows
- production-style RAG ingestion architecture
- improved chunk quality and semantic coherence

## Resources and References Used

- Groq API Documentation
- Mistral AI Documentation
- Sentence Transformers Documentation
- FastAPI Documentation
- MongoDB Documentation
- Retrieval-Augmented Generation (RAG) Concepts
- Vector Embedding Retrieval Concepts
- Semantic Search Architecture Concepts
- Prompt Engineering Concepts

## Concepts Learned

**Hierarchical Semantic Chunking**

Learned how production-grade RAG systems avoid processing entire documents in a single request by implementing multi-stage chunking workflows.

Implemented:

- rough semantic preprocessing
- paragraph-aware segmentation

- hierarchical chunk generation
- Groq-powered semantic splitting
- semantic boundary refinement

Understood:

- retrieval-oriented chunking
- semantic coherence preservation
- topic-aware chunk segmentation
- contextual retrieval optimization

**Groq AI Semantic Chunking**

Learned how large language models can intelligently segment documents into semantically meaningful retrieval units.

Implemented:

- Groq API integration
- semantic chunking prompts
- retrieval-focused chunk generation
- contextual chunk validation
- semantic cleanup filtering

Understood:

- LLM-driven preprocessing pipelines
- AI-assisted ingestion workflows
- semantic segmentation prompting
- retrieval-oriented chunk generation

**Retrieval Precision Optimization**

Improved the semantic retrieval quality significantly.

Previous retrieval behavior:

- vague contextual matches
- weak chunk relevance
- oversized retrieval contexts
- fragmented semantic retrieval

New retrieval behavior:

- focused contextual chunks
- topic-specific retrieval
- improved conversational grounding
- cleaner semantic similarity matching

Implemented:

- chunk cleanup filtering
- malformed fragment removal
- contextual chunk validation
- retrieval-friendly chunk sizing

**Persistent Knowledge Base Improvements**

Improved MongoDB chunk persistence workflows.

Implemented:

- overwrite-based chunk updates
- duplicate prevention improvements
- synchronized vector persistence
- retrieval memory reset workflows

Understood:

- persistent vector database workflows
- document update handling
- semantic synchronization
- knowledge base lifecycle management

## Libraries, Modules, and Components Learned

- **Groq SDK:** Used for AI-powered semantic chunk generation.
- **Sentence Transformers:** Used for semantic embedding generation.
- **FastAPI:** Used for backend API development.
- **MongoDB:** Used for vector storage and chunk persistence.
- **Mistral AI SDK:** Used for contextual conversational response generation.
- **dotenv:** Used for API key management.
- **cosine_similarity:** Used for semantic similarity scoring.
- **JSON parsing workflows:** Used for AI-generated chunk handling.

## Practical Implementation

- hierarchical semantic chunking architecture
- paragraph-aware document preprocessing
- Groq semantic chunk generation pipeline
- contextual chunk validation logic
- malformed fragment cleanup
- overwrite-based MongoDB updates
- improved semantic retrieval precision
- conversational follow-up retrieval handling
- retrieval-oriented chunk refinement
- production-style RAG ingestion workflow
- improved contextual response grounding
- cleaner semantic segmentation

## Architecture Understood

Document Upload
↓
Text Extraction (TXT / PDF / OCR)
↓
Paragraph-Aware Preprocessing
↓
Hierarchical Semantic Sectioning
↓
Groq AI Semantic Chunking
↓
Chunk Cleanup & Validation
↓
SentenceTransformer Embedding Generation
↓
MongoDB Vector Storage
↓
Semantic Similarity Retrieval

↓

Mistral AI Context Injection

↓

Human-Like Conversational Response

↓

Frontend Chatbot Rendering

## Challenges Faced

**Groq Model Deprecation Issues**

Encountered:

- deprecated model errors
- upload failures
- API request failures

Resolved by:

- upgrading to supported Groq models
- correcting model configurations
- updating API request handling

**Token Limit Problems**

Encountered:

- oversized document requests
- Groq token overflow errors
- failed semantic chunk generation

Resolved through:

- hierarchical preprocessing
- paragraph-aware segmentation
- multi-stage chunk generation

**Heading-Only Chunk Generation**

Initial semantic chunking produced:

- titles only
- weak contextual information
- unusable retrieval chunks

Resolved by:

- prompt engineering improvements
- contextual chunk enforcement
- retrieval-oriented chunk instructions

**Giant Full-Document Chunk Problems**

Encountered:

- oversized semantic chunks
- poor retrieval precision
- weak semantic targeting

Resolved through:

- semantic segmentation rules
- topic-aware chunk splitting
- paragraph-aware preprocessing

**Broken Chunk Boundary Issues**

Discovered:

- malformed sentence fragments
- cut-off chunk endings
- incomplete contextual sections

Resolved by:

- paragraph-aware splitting
- cleanup filtering
- malformed fragment removal

## Conclusion

Today's work transformed the chatbot into a significantly more advanced production-style RAG architecture with intelligent semantic chunking and improved contextual retrieval quality.

The platform now supports:

- hierarchical semantic chunking
- contextual retrieval workflows
- production-style RAG ingestion
- AI-powered semantic segmentation
- persistent vector knowledge management
- conversational follow-up retrieval
- retrieval-focused semantic chunking
- cleaner semantic embeddings
- improved contextual grounding

The concepts learned today form the foundation of:

- production-grade RAG systems
- semantic search architectures
- enterprise AI retrieval systems
- vector database applications
- AI ingestion pipelines
- intelligent document retrieval systems
- conversational AI assistants

## Prepared By

### Divesh Kumar S